

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«СЕВЕРО-КАВКАЗСКИЙ ФЕДЕРАЛЬНЫЙ УНИВЕРСИТЕТ»

Институт перспективной инженерии
Департамент цифровых, робототехнических систем и электроники

ОТЧЕТ
ПО ЛАБОРАТОРНОЙ РАБОТЕ №5
дисциплины «Искусственный интеллект и машинное обучение»

Вариант 20

Выполнил:
Скоробогатов Виктор Андреевич
2 курс, группа ИВТ-б-о-24-1,
09.03.01 «Информатика и
вычислительная техника»,
направленность (профиль)
«Программное обеспечение средств
вычислительной техники и
автоматизированных систем», очная
форма обучения

(подпись)

Руководитель практики:
Воронкин Роман Александрович,
доцент департамента цифровых,
робототехнических систем и
электроники института перспективной
инженерии

(подпись)

Отчет защищен с оценкой _____ Дата защиты _____

Ставрополь, 2026 г.

Тема: Введение в pandas: изучение структуры DataFrame и базовых операций

Цель: познакомить с основами работы с библиотекой pandas, в частности, со структурой данных DataFrame

Ход работы

Ссылка на репозиторий:

<https://github.com/LostsSs78/AI-Lab5>

Для выполнения работы был создан новый общедоступный репозиторий на GitHub, использующий лицензию MIT и язык программирования Python, далее он был клонирован на персональный компьютер, также был дополнен файл .gitignore необходимыми правилами для языка Python, IDE PyCharm и Jupyter Lab:

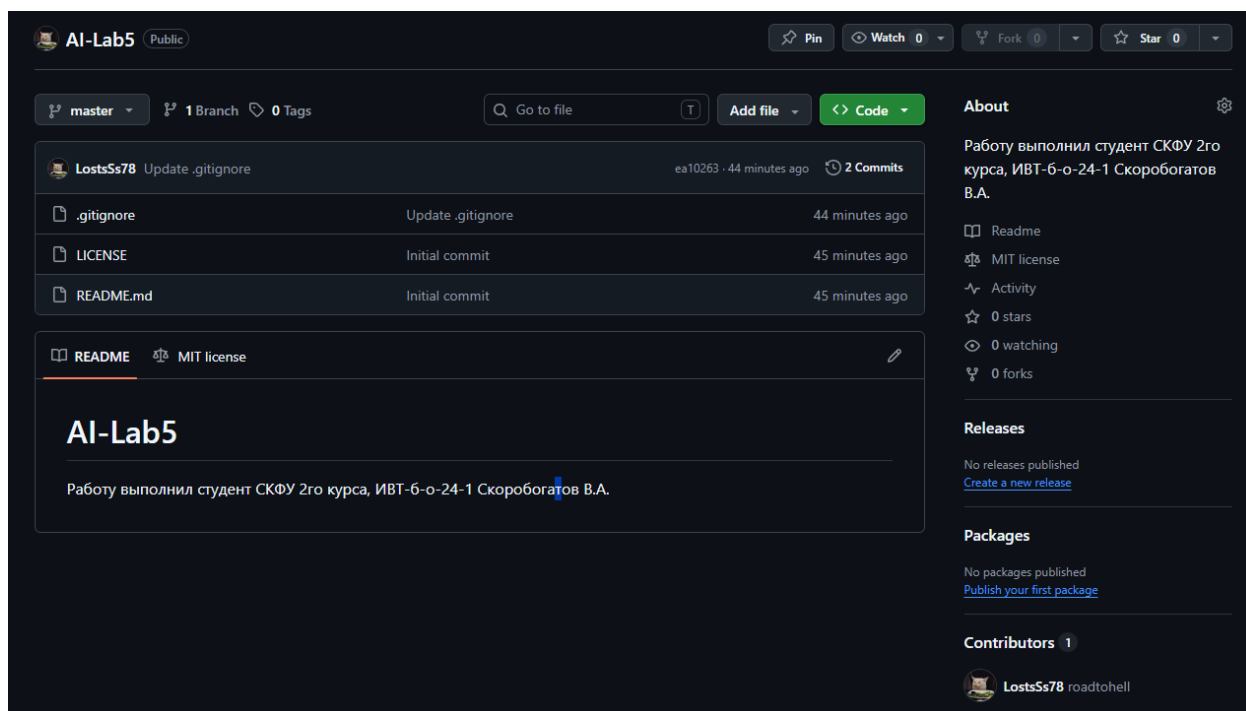


Рисунок 1. Репозиторий на GitHub

Далее были проработаны примеры, приведённые в лабораторной работе:

Пример 1

```
import pandas as pd
import numpy as np

data = {
    "Возраст": [25, 30, 22],
    "Город": ["Москва", "СПб", "Казань"]
}

df = pd.DataFrame(data, index=["Анна", "Иван", "Ольга"])
print(df)
```

	Возраст	Город
Анна	25	Москва
Иван	30	СПб
Ольга	22	Казань

Рисунок 2. Пример 1

Пример 2

```
ages = pd.Series([25, 30, 22], index=["a", "b", "c"])
cities = pd.Series(["Москва", "СПб", "Казань"], index=["a", "b", "c"])

df = pd.DataFrame({"Возраст": ages, "Город": cities})
print(df)
```

	Возраст	Город
a	25	Москва
b	30	СПб
c	22	Казань

Рисунок 3. Пример 2

```

data = np.array([
    ["Анна", 25, "Москва"],
    ["Иван", 30, "СПб"],
    ["Ольга", 22, "Казань"]
])

df = pd.DataFrame(data, columns=["Имя", "Возраст", "Город"])

data = {
    "Имя": ["Анна", "Иван", "Ольга", "Петр", "Мария", "Сергей"],
    "Возраст": [25, 30, 22, 40, 35, 28],
    "Город": ["Москва", "СПб", "Казань", "Новосибирск", "Екатеринбург", "Сочи"]
}

df = pd.DataFrame(data)
print(df.head(3))
print("-"*20)

print(df.tail(2))
print("-"*20)

df.info()
print("-"*20)

print(df.describe())

```

Рисунок 4. Пример 2

```

      Имя  Возраст  Город
0  Анна      25  Москва
1  Иван      30   СПб
2  Ольга     22  Казань
-----
      Имя  Возраст      Город
4  Мария      35  Екатеринбург
5  Сергей     28           Сочи
-----
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 6 entries, 0 to 5
Data columns (total 3 columns):
#   Column  Non-Null Count  Dtype
---  -
0   Имя     6 non-null      object
1   Возраст  6 non-null      int64
2   Город    6 non-null      object
dtypes: int64(1), object(2)
memory usage: 168.0+ bytes
-----
      Возраст
count    6.00000
mean    30.00000
std      6.60303
min     22.00000
25%     25.75000
50%     29.00000
75%     33.75000
max     40.00000

```

Рисунок 5. Результат выполнения примера 2

Пример 3

```
data = {
    "Имя": ["Анна", "Иван", "Ольга", "Петр"],
    "Возраст": [25, 30, 22, 40],
    "Город": ["Москва", "СПб", "Казань", "Новосибирск"]
}

df = pd.DataFrame(data, index=["a", "b", "c", "d"])
print(df)

print(df.loc["b"]) # Доступ к строке "b"
print("-"*20)

print(df.loc["c", "Город"]) # Доступ к конкретному значению (город Ольги)
print("-"*20)

print(df.loc[["a", "c"]]) # Выбор строк "a" и "c"
print("-"*20)

print(df.loc[:, ["Имя", "Город"]]) # Выбор всех строк, но только двух столбцов
print("-"*20)

print(df.loc[df["Возраст"] > 25]) # Фильтрация: возраст больше 25
```

Рисунок 6. Пример 3

Пример 4

```
data = {
    "Имя": ["Анна", "Иван", "Ольга"],
    "Возраст": [25, 30, 22]
}

df = pd.DataFrame(data)
df["Город"] = ["Москва", "СПб", "Казань"] # Добавляем новый столбец

print(df)
df["Страна"] = "Россия"
print("-"*20)

new_data = pd.DataFrame([{"Имя": "Алексей", "Возраст": 29, "Город": "Томск", "Страна": "Россия"}])

df = pd.concat([df, new_data], ignore_index=True)
print(df)
print("-"*20)

new_rows = pd.DataFrame([
    {"Имя": "Дмитрий", "Возраст": 31, "Город": "Самара", "Страна": "Россия"},
    {"Имя": "Елена", "Возраст": 27, "Город": "Воронеж", "Страна": "Россия"}
])

df = pd.concat([df, new_rows], ignore_index=True)
print(df)
```

Рисунок 7. Пример 4

Пример 5

```
data = {
    "Имя": ["Анна", "Иван", "Ольга"],
    "Возраст": [25, 30, 22],
    "Город": ["Москва", "СПб", "Казань"],
    "Зарплата": [50000, 60000, 45000]
}

df = pd.DataFrame(data)

df = df.drop(columns=["Зарплата"])
print(df)
print("-"*20)

df = df.drop(columns=["Возраст", "Город"])
print(df)
print("-"*20)

df["Имя"] = ["Анна", "Иван", "Ольга"] # Вернем столбец для примера
del df["Имя"]
print(df)
print("-"*20)

df["Имя"] = ["Анна", "Иван", "Ольга"] # Вернем столбец для примера
удаленный_столбец = df.pop("Имя")
print(удаленный_столбец)
```

Рисунок 8. Пример 5

```
data = {
    "Имя": ["Анна", "Иван", "Ольга"],
    "Возраст": [25, 30, 22],
    "Город": ["Москва", "СПб", "Казань"],
    "Зарплата": [50000, 60000, 45000]
}

df = pd.DataFrame(data)

df = df.drop(index=1)
print(df)
print("-"*20)

df = df.drop(index=[0, 2])
print(df)
print("-"*20)

df = pd.DataFrame(data) # Восстанавливаем DataFrame
df = df[df["Возраст"] > 25]
print(df)
print("-"*20)

df = pd.DataFrame(data) # Восстанавливаем DataFrame
df.loc[3] = ["Мария", None, "Самара", 55000] # Добавим строку с пропуском
df = df.dropna() # Удалил все строки с пропущенными значениями
print(df)
print("-"*20)

df = pd.DataFrame({
    "Имя": ["Анна", "Иван", "Анна", "Ольга"],
    "Возраст": [25, 30, 25, 22]
})

df = df.drop_duplicates()
print(df)
```

Рисунок 9. Пример 6

Пример 7

```
data = {
    "Имя": ["Анна", "Иван", "Ольга", "Петр", "Мария"],
    "Возраст": [25, 30, 22, 40, 35],
    "Город": ["Москва", "СПб", "Казань", "Новосибирск", "СПб"],
    "Зарплата": [50000, 60000, 45000, 70000, 65000]
}

df = pd.DataFrame(data)
print(df)
print("-"*20)

df_filtered = df.query("Возраст > 30")
print(df_filtered)
print("-"*20)

df_filtered = df.query("Город == 'СПб' and Зарплата > 60000")
print(df_filtered)
print("-"*20)

min_age = 25
df_filtered = df.query("Возраст > @min_age")
print(df_filtered)
```

Рисунок 10. Пример 7

Пример 8

```
data = {
    "Имя": ["Анна", "Иван", "Ольга", "Петр", "Мария"],
    "Возраст": [25, 30, 22, 40, 35],
    "Город": ["Москва", "СПб", "Казань", "Новосибирск", "СПб"],
    "Зарплата": [50000, 60000, 45000, 70000, 65000]
}

df = pd.DataFrame(data)

df_filtered = df[df["Город"].isin(["Москва", "СПб"])]
print(df_filtered)
print("-"*20)

df_filtered = df[~df["Город"].isin(["Москва", "СПб"])]
print(df_filtered)
print("-"*20)

df_filtered = df[df["Возраст"].between(25, 35)]
print(df_filtered)
print("-"*20)

df_filtered = df[df["Возраст"].between(25, 35, inclusive="neither")]
print(df_filtered)
```

Рисунок 11. Пример 8

Пример 9

```
data = {
    "Имя": ["Анна", "Иван", "Ольга", "Петр", None],
    "Возраст": [25, 30, 22, None, 35],
    "Город": ["Москва", "СПб", "Казань", "Новосибирск", "СПб"]
}

df = pd.DataFrame(data)
print(df)
print("-"*20)

print(df.count())
print("-"*20)

print(df["Город"].value_counts())
print("-"*20)

print(df.nunique())
print("-"*20)

print(df.isna().sum())
```

Рисунок 12. Пример 9

Пример 10

```
data = {
    "Имя": ["Анна", "Иван", "Ольга", "Петр", "Мария"],
    "Возраст": [25, 30, 22, 40, 35],
    "Зарплата": [50000, 60000, 45000, 70000, 65000]
}

df = pd.DataFrame(data)
print(df)
print("-"*20)

df_sorted = df.sort_values(by="Возраст")
print(df_sorted)
print("-"*20)

df_sorted = df.sort_values(by=["Возраст", "Зарплата"], ascending=[True, False])
print(df_sorted)
print("-"*20)

df.loc[5] = ["Елена", None, 55000] # Добавляем строку с NaN в "Возраст"
df_sorted = df.sort_values(by="Возраст", na_position="first")
print(df_sorted)
```

Рисунок 13. Пример 10

Пример 11

```
data = {
    "Имя": ["Анна", "Иван", "Ольга", "Петр", "Мария"],
    "Возраст": [25, 30, 22, 40, 35],
    "Зарплата": [50000, 60000, 45000, 70000, 65000]
}

df = pd.DataFrame(data)

# Добавим строку с NaN для демонстрации
df.loc[5] = ["Елена", None, 55000]

df_sorted = df.sort_index()
print(df_sorted)
print("-"*20)

df_sorted = df.sort_index(ascending=False)
print(df_sorted)
```

[26]

Рисунок 14. Пример 11

Пример 12

```
from IPython.display import display

df = pd.DataFrame({
    "Имя": ["Анна", "Иван", "Ольга"],
    "Возраст": [25, 30, 22],
    "Город": ["Москва", "СПб", "Казань"]
})

print(df)
display(df)
```

[27]

Рисунок 15. Пример 12

Далее были выполнены задания лабораторной работы, были созданы необходимые файлы и подготовлены данные для выполнения работы:

```

from pathlib import Path
from IPython.display import display

import numpy as np
import pandas as pd

pd.set_option("display.max_columns", None)
pd.set_option("display.width", 120)

DATA_DIR = Path("practice_data")
DATA_DIR.mkdir(exist_ok=True)

EMPLOYEE_RECORDS = [
    {
        "ID": 1,
        "Имя": "Иван",
        "Возраст": 25,
        "Должность": "Инженер",
        "Отдел": "IT",
        "Зарплата": 60000,
        "Стаж работы": 2,
    },
    {
        "ID": 2,
        "Имя": "Ольга",
        "Возраст": 30,
        "Должность": "Аналитик",
        "Отдел": "Маркетинг",
        "Зарплата": 75000,
        "Стаж работы": 5,
    },
]

employees_df = pd.DataFrame(EMPLOYEE_RECORDS)
clients_df = pd.DataFrame(CLIENT_RECORDS, columns=CLIENT_COLUMNS)
clients_missing_df = pd.DataFrame(
    MISSING_CLIENT_RECORDS,
    columns=CLIENT_COLUMNS,
)

display(employees_df.head())
display(clients_df.head())
display(clients_missing_df.head())

```

Рисунок 16. Подготовка данных

Задание 1. Создание "DataFrame" разными способами

****Условие.****

1. Создайте "DataFrame" из словаря списков с данными о пяти сотрудниках, взяв данные из таблицы 1.
2. Создайте "DataFrame" из списка словарей с теми же данными.
3. Создайте "DataFrame" из массива "NumPy", заполненного случайными числами от 20 до 60 для возраста сотрудников.
4. Проверьте типы данных в каждом столбце с помощью ".info()".

```
first_five_employees = EMPLOYEE_RECORDS[:5]

employee_dict = {
    column: [row[column] for row in first_five_employees]
    for column in employees_df.columns
}
df_from_dict = pd.DataFrame(employee_dict)

print("DataFrame из словаря списков:")
display(df_from_dict)

print("Информация о DataFrame из словаря списков:")
df_from_dict.info()
```

Рисунок 17. Задание 1

DataFrame из словаря списков:

Информация о DataFrame из словаря списков:

<class 'pandas.core.frame.DataFrame'>

RangeIndex: 5 entries, 0 to 4

Data columns (total 7 columns):

#	Column	Non-Null Count	Dtype
0	ID	5 non-null	int64
1	Имя	5 non-null	object
2	Возраст	5 non-null	int64
3	Должность	5 non-null	object
4	Отдел	5 non-null	object
5	Зарплата	5 non-null	int64
6	Стаж работы	5 non-null	int64

dtypes: int64(4), object(3)

memory usage: 412.0+ bytes

	ID	Имя	Возраст	Должность	Отдел	Зарплата	Стаж работы
0	1	Иван	25	Инженер	IT	60000	2
1	2	Ольга	30	Аналитик	Маркетинг	75000	5
2	3	Алексей	40	Менеджер	Продажи	90000	15
3	4	Мария	35	Программист	IT	80000	7
4	5	Сергей	28	Специалист	HR	50000	3

Рисунок 17.1. Задание 1

```
df_from_records = pd.DataFrame(first_five_employees)

print("DataFrame из списка словарей:")
display(df_from_records)

print("Информация о DataFrame из списка словарей:")
df_from_records.info()
```

DataFrame из списка словарей:

Информация о DataFrame из списка словарей:

```
<class 'pandas.core.frame.DataFrame'>
```

```
RangeIndex: 5 entries, 0 to 4
```

```
Data columns (total 7 columns):
```

```
#   Column      Non-Null Count  Dtype
---  -
0   ID          5 non-null      int64
1   Имя          5 non-null      object
2   Возраст      5 non-null      int64
3   Должность    5 non-null      object
4   Отдел         5 non-null      object
5   Зарплата     5 non-null      int64
6   Стаж работы  5 non-null      int64
```

```
dtypes: int64(4), object(3)
```

```
memory usage: 412.0+ bytes
```

	ID	Имя	Возраст	Должность	Отдел	Зарплата	Стаж работы
0	1	Иван	25	Инженер	IT	60000	2
1	2	Ольга	30	Аналитик	Маркетинг	75000	5
2	3	Алексей	40	Менеджер	Продажи	90000	15
3	4	Мария	35	Программист	IT	80000	7
4	5	Сергей	28	Специалист	HR	50000	3

Рисунок 17.2. Задание 1

```

rng = np.random.default_rng(seed=42)
random_ages = rng.integers(low=20, high=61, size=(5, 1))
df_from_numpy = pd.DataFrame(random_ages, columns=["Возраст"])

print("DataFrame из массива NumPy со случайными возрастaми:")
display(df_from_numpy)

print("Информация о DataFrame из массива NumPy:")
df_from_numpy.info()

```

DataFrame из массива NumPy со случайными возрастaми:

Информация о DataFrame из массива NumPy:

<class 'pandas.core.frame.DataFrame'>

RangeIndex: 5 entries, 0 to 4

Data columns (total 1 columns):

#	Column	Non-Null Count	Dtype
---	--------	----------------	-------

0	Возраст	5 non-null	int64
---	---------	------------	-------

dtypes: int64(1)

memory usage: 172.0 bytes

	Возраст
0	23
1	51
2	46
3	37
4	37

Рисунок 17.3. Задание 1

Задание 2. Чтение данных из файлов ("CSV", "Excel", "JSON")

****Условие.****

Используйте предоставленные таблицы и сохраните их в файлы перед чтением.

1. Сохраните таблицу 1 в "CSV" и затем загрузите ее обратно в "DataFrame".
2. Запишите таблицу 2 в "Excel" ("data.xlsx", лист "Клиенты") и прочитайте ее в "DataFrame".
3. Экспортируйте таблицу 1 в формат "JSON" и затем прочитайте ее обратно в "DataFrame".

Выведите первые 5 строк загруженных "DataFrame".

```
csv_path = DATA_DIR / "employees.csv"
excel_path = DATA_DIR / "data.xlsx"
json_path = DATA_DIR / "employees.json"

employees_df.to_csv(csv_path, index=False, encoding="utf-8-sig")
clients_df.to_excel(excel_path, sheet_name="Клиенты", index=False)
employees_df.to_json(json_path, orient="records", force_ascii=False)

employees_from_csv = pd.read_csv(csv_path)
clients_from_excel = pd.read_excel(excel_path, sheet_name="Клиенты")
employees_from_json = pd.read_json(json_path, orient="records")

print("Первые 5 строк таблицы 1, загруженной из CSV:")
display(employees_from_csv.head())

print("Первые 5 строк таблицы 2, загруженной из Excel:")
display(clients_from_excel.head())

print("Первые 5 строк таблицы 1, загруженной из JSON:")
display(employees_from_json.head())
```

Рисунок 18. Задание 2

Первые 5 строк таблицы 1, загруженной из CSV:

Первые 5 строк таблицы 2, загруженной из Excel:

Первые 5 строк таблицы 1, загруженной из JSON:

	ID	Имя	Возраст	Должность	Отдел	Зарплата	Стаж работы
0	1	Иван	25	Инженер	IT	60000	2
1	2	Ольга	30	Аналитик	Маркетинг	75000	5
2	3	Алексей	40	Менеджер	Продажи	90000	15
3	4	Мария	35	Программист	IT	80000	7
4	5	Сергей	28	Специалист	HR	50000	3

	ID	Имя	Возраст	Город	Баланс на счете	Кредитная история
0	1	Иван	34	Москва	120000	Хорошая
1	2	Ольга	27	Санкт-Петербург	80000	Средняя
2	3	Алексей	45	Казань	150000	Плохая
3	4	Мария	38	Новосибирск	200000	Хорошая
4	5	Сергей	29	Екатеринбург	95000	Средняя

	ID	Имя	Возраст	Должность	Отдел	Зарплата	Стаж работы
0	1	Иван	25	Инженер	IT	60000	2
1	2	Ольга	30	Аналитик	Маркетинг	75000	5
2	3	Алексей	40	Менеджер	Продажи	90000	15
3	4	Мария	35	Программист	IT	80000	7
4	5	Сергей	28	Специалист	HR	50000	3

Рисунок 18.1 Задание 2

Задание 3. Доступ к данным (".loc", ".iloc", ".at", ".iat")

****Условие.****

Используя таблицу 1:

1. Получите информацию о сотруднике с "ID = 5" с помощью ".loc[]".
2. Выведите возраст третьего сотрудника в таблице с ".iloc[]".
3. Выведите название отдела для сотрудника "Мария" с ".at[]".
4. Выведите зарплату сотрудника, находящегося в четвертой строке и пятом столбце, используя ".iat[]".

Объясните разницу между ".loc[]", ".iloc[]", ".at[]", ".iat[]".

```
employees_by_id = employees_df.set_index("ID")

employee_id_5 = employees_by_id.loc[5]
third_employee_age = employees_by_id.iloc[2]["Возраст"]
maria_id = employees_by_id.index[employees_by_id["Имя"] == "Мария"][0]
maria_department = employees_by_id.at[maria_id, "Отдел"]
fourth_row_fifth_column_salary = employees_by_id.iat[3, 4]

print("Сотрудник с ID = 5:")
display(employee_id_5.to_frame().T)

print(f"Возраст третьего сотрудника: {third_employee_age}")
print(f"Отдел сотрудника Мария: {maria_department}")
print(
    "Зарплата в четвертой строке и пятом столбце: "
    f"{fourth_row_fifth_column_salary}"
)
```

Рисунок 19. Задание 3

Сотрудник с ID = 5:

Возраст третьего сотрудника: 40

Отдел сотрудника Мария: IT

Зарплата в четвертой строке и пятом столбце: 80000

	Имя	Возраст	Должность	Отдел	Зарплата	Стаж работы
5	Сергей	28	Специалист	HR	50000	3

Рисунок 19.1. Задание 3

Задание 4. Добавление новых столбцов и строк

****Условие.****

Используя таблицу 1:

1. Добавьте новый столбец "Категория зарплаты": "Низкая", если зарплата меньше 60000; "Средняя", если зарплата от 60000 до 100000; "Высокая", если зарплата не меньше 100000.
2. Добавьте нового сотрудника с данными: "Антон", 32, "Разработчик", "IT", 85000, 6.
3. Добавьте двух новых сотрудников одновременно, используя "pd.concat()".

Выведите обновленный "DataFrame".

```
def get_salary_category(salary):
    """Return a text category for an employee salary."""
    if salary < 60000:
        return "Низкая"
    if salary < 100000:
        return "Средняя"
    return "Высокая"

updated_employees = employees_by_id.copy()
updated_employees["Категория зарплаты"] = updated_employees[
    "Зарплата"
].apply(get_salary_category)

print("После добавления столбца 'Категория зарплаты':")
display(updated_employees)
```

Рисунок 20. Задание 4

После добавления столбца 'Категория зарплаты':

	Имя	Возраст	Должность	Отдел	Зарплата	Стаж работы	Категория зарплаты
ID							
1	Иван	25	Инженер	IT	60000	2	Средняя
2	Ольга	30	Аналитик	Маркетинг	75000	5	Средняя
3	Алексей	40	Менеджер	Продажи	90000	15	Средняя
4	Мария	35	Программист	IT	80000	7	Средняя
5	Сергей	28	Специалист	HR	50000	3	Низкая
6	Анна	32	Разработчик	IT	85000	6	Средняя
7	Дмитрий	45	HR	HR	48000	12	Низкая
8	Елена	29	Маркетолог	Маркетинг	70000	4	Средняя
9	Виктор	31	Юрист	Юридический	95000	10	Средняя
10	Алиса	27	Дизайнер	Дизайн	62000	5	Средняя
11	Павел	33	Администратор	Администрация	55000	7	Низкая
12	Светлана	26	Тестировщик	Тестирование	67000	2	Средняя
13	Роман	42	Финансист	Финансы	105000	20	Высокая
14	Татьяна	37	Редактор	Редакция	72000	9	Средняя
15	Николай	39	Логист	Логистика	75000	11	Средняя
16	Валерия	24	SEO-специалист	SEO	64000	3	Средняя
17	Григорий	50	Бухгалтер	Бухгалтерия	110000	25	Высокая
18	Юлия	45	Директор	Финансы	150000	20	Высокая
19	Степан	41	Экономист	Экономика	98000	14	Средняя
20	Василиса	38	Проект-менеджер	Продажи	88000	8	Средняя

Рисунок 20.1 Задание 4

```
anton_salary = 85000
updated_employees.loc[21] = {
    "Имя": "Антон",
    "Возраст": 32,
    "Должность": "Разработчик",
    "Отдел": "IT",
    "Зарплата": anton_salary,
    "Стаж работы": 6,
    "Категория зарплаты": get_salary_category(anton_salary),
}

print("После добавления сотрудника Антон:")
display(updated_employees.tail())
```

После добавления сотрудника Антон:

	Имя	Возраст	Должность	Отдел	Зарплата	Стаж работы	Категория зарплаты
ID							
17	Григорий	50	Бухгалтер	Бухгалтерия	110000	25	Высокая
18	Юлия	45	Директор	Финансы	150000	20	Высокая
19	Степан	41	Экономист	Экономика	98000	14	Средняя
20	Василиса	38	Проект-менеджер	Продажи	88000	8	Средняя
21	Антон	32	Разработчик	IT	85000	6	Средняя

Рисунок 20.2. Задание 4

```
new_employee_records = [
    {
        "Имя": "Кирилл",
        "Возраст": 29,
        "Должность": "Аналитик данных",
        "Отдел": "IT",
        "Зарплата": 92000,
        "Стаж работы": 4,
    },
    {
        "Имя": "Евгения",
        "Возраст": 36,
        "Должность": "Руководитель проекта",
        "Отдел": "Продажи",
        "Зарплата": 115000,
        "Стаж работы": 9,
    },
]

new_employees = pd.DataFrame(new_employee_records, index=[22, 23])
new_employees["Категория зарплаты"] = new_employees["Зарплата"].apply(
    get_salary_category
)

new_employees.index.name = "ID"
updated_employees = pd.concat([updated_employees, new_employees])

print("После добавления двух новых сотрудников через pd.concat():")
display(updated_employees)
```

Рисунок 20.3. Задание 4

	Имя	Возраст	Должность	Отдел	Зарплата	Стаж работы	Категория зарплаты
ID							
1	Иван	25	Инженер	IT	60000	2	Средняя
2	Ольга	30	Аналитик	Маркетинг	75000	5	Средняя
3	Алексей	40	Менеджер	Продажи	90000	15	Средняя
4	Мария	35	Программист	IT	80000	7	Средняя
5	Сергей	28	Специалист	HR	50000	3	Низкая
6	Анна	32	Разработчик	IT	85000	6	Средняя
7	Дмитрий	45	HR	HR	48000	12	Низкая
8	Елена	29	Маркетолог	Маркетинг	70000	4	Средняя
9	Виктор	31	Юрист	Юридический	95000	10	Средняя
10	Алиса	27	Дизайнер	Дизайн	62000	5	Средняя
11	Павел	33	Администратор	Администрация	55000	7	Низкая
12	Светлана	26	Тестировщик	Тестирование	67000	2	Средняя
13	Роман	42	Финансист	Финансы	105000	20	Высокая
14	Татьяна	37	Редактор	Редакция	72000	9	Средняя
15	Николай	39	Логист	Логистика	75000	11	Средняя
16	Валерия	24	SEO-специалист	SEO	64000	3	Средняя
17	Григорий	50	Бухгалтер	Бухгалтерия	110000	25	Высокая
18	Юлия	45	Директор	Финансы	150000	20	Высокая
19	Степан	41	Экономист	Экономика	98000	14	Средняя
20	Василиса	38	Проект-менеджер	Продажи	88000	8	Средняя
21	Антон	32	Разработчик	IT	85000	6	Средняя
22	Кирилл	29	Аналитик данных	IT	92000	4	Средняя
23	Евгения	36	Руководитель проекта	Продажи	115000	9	Высокая

Рисунок 20.4. Задание 4

Задание 5. Удаление строк и столбцов

****Условие.****

Используя таблицу 1:

1. Удалите столбец "Категория зарплаты" с `".drop()"`.
2. Удалите строку с `"ID = 10"`.
3. Удалите все строки, где `"Стаж работы < 3"`.
4. Удалите все столбцы, кроме `"Имя", "Должность", "Зарплата"`.

Выведите `"DataFrame"` после каждого шага.

```
deleted_data = updated_employees.copy()

deleted_data = deleted_data.drop(columns=["Категория зарплаты"])
print("Шаг 1. После удаления столбца 'Категория зарплаты':")
display(deleted_data)

deleted_data = deleted_data.drop(index=10)
print("Шаг 2. После удаления строки с ID = 10:")
display(deleted_data)

deleted_data = deleted_data[deleted_data["Стаж работы"] >= 3]
print("Шаг 3. После удаления строк, где стаж работы меньше 3 лет:")
display(deleted_data)

deleted_data = deleted_data[["Имя", "Должность", "Зарплата"]]
print("Шаг 4. После удаления всех столбцов, кроме нужных:")
display(deleted_data)
```

Рисунок 21. Задание 5

Задание 6. Фильтрация данных ("query", "isin", "between")

****Условие.****

Используя таблицу 2:

1. Выберите всех клиентов из "Москва" или "Санкт-Петербург", используя ".isin()".
2. Выберите клиентов, у которых "Баланс на счете" от 100000 до 250000, используя ".between()".
3. Отфильтруйте клиентов, у которых "Кредитная история" – "Хорошая" и "Баланс на счете" > 150000, используя ".query()".

Выведите результаты каждого фильтра.

```
selected_cities = ["Москва", "Санкт-Петербург"]
clients_from_selected_cities = clients_df[
    clients_df["Город"].isin(selected_cities)
]

clients_with_balance_range = clients_df[
    clients_df["Баланс на счете"].between(100000, 250000)
]

clients_good_credit = clients_df.query(
    "`Кредитная история` == 'Хорошая' and `Баланс на счете` > 150000"
)

print("Клиенты из Москвы или Санкт-Петербурга:")
display(clients_from_selected_cities)

print("Клиенты с балансом от 100000 до 250000:")
display(clients_with_balance_range)

print("Клиенты с хорошей кредитной историей и балансом больше 150000:")
display(clients_good_credit)
```

Рисунок 22. Задание 6

Клиенты из Москвы или Санкт-Петербурга:

Клиенты с балансом от 100000 до 250000:

Клиенты с хорошей кредитной историей и балансом больше 150000:

	ID	Имя	Возраст	Город	Баланс на счете	Кредитная история
0	1	Иван	34	Москва	120000	Хорошая
1	2	Ольга	27	Санкт-Петербург	80000	Средняя

	ID	Имя	Возраст	Город	Баланс на счете	Кредитная история
0	1	Иван	34	Москва	120000	Хорошая
2	3	Алексей	45	Казань	150000	Плохая
3	4	Мария	38	Новосибирск	200000	Хорошая
6	7	Дмитрий	31	Челябинск	140000	Средняя
7	8	Елена	40	Краснодар	175000	Хорошая
8	9	Виктор	28	Ростов-на-Дону	110000	Плохая
10	11	Павел	46	Омск	250000	Хорошая
11	12	Светлана	37	Пермь	210000	Отличная
12	13	Роман	41	Тюмень	135000	Средняя
13	14	Татьяна	25	Саратов	155000	Хорошая
14	15	Николай	39	Самара	125000	Средняя
15	16	Валерия	42	Волгоград	180000	Плохая
18	19	Степан	30	Хабаровск	105000	Средняя

	ID	Имя	Возраст	Город	Баланс на счете	Кредитная история
3	4	Мария	38	Новосибирск	200000	Хорошая
7	8	Елена	40	Краснодар	175000	Хорошая
10	11	Павел	46	Омск	250000	Хорошая
13	14	Татьяна	25	Саратов	155000	Хорошая
17	18	Юлия	50	Иркутск	320000	Хорошая

Рисунок 22.1 Задание 6

Задание 7. Подсчет значений ("count", "value_counts", "nunique")

****Условие.****

Используя таблицу 2:

1. Подсчитайте количество непустых значений в каждом столбце (`".count()"`).
2. Определите частоту встречаемости значений в "Город" (`".value_counts()"`).
3. Найдите количество уникальных значений в "Город", "Возраст", "Баланс на счете" (`".nunique()"`).

Выведите результаты.

```
non_empty_counts = clients_df.count()
city_value_counts = clients_df["Город"].value_counts()
unique_counts = clients_df[[
    "Город",
    "Возраст",
    "Баланс на счете",
]].nunique()

print("Количество непустых значений в каждом столбце:")
display(non_empty_counts.to_frame(name="Количество"))

print("Частота встречаемости значений в столбце 'Город':")
display(city_value_counts.to_frame(name="Частота"))

print("Количество уникальных значений:")
display(unique_counts.to_frame(name="Уникальных значений"))
```

Рисунок 23. Задание 7

	Количество
ID	20
Имя	20
Возраст	20
Город	20
Баланс на счете	20
Кредитная история	20

	Частота
Город	
Москва	1
Санкт-Петербург	1
Хабаровск	1
Иркутск	1
Барнаул	1
Волгоград	1
Самара	1
Саратов	1
Тюмень	1
Пермь	1
Омск	1
Уфа	1
Ростов-на-Дону	1
Краснодар	1
Челябинск	1
Воронеж	1
Екатеринбург	1
Новосибирск	1
Казань	1
Томск	1

	Уникальных значений
Город	20
Возраст	19
Баланс на счете	20

Рисунок 23.1. Задание 7

Задание 8. Обнаружение пропусков ("isna", "notna")

****Условие.****

Используя таблицу 3:

1. Подсчитайте количество "NaN" в каждом столбце (`".isna().sum()"`).
2. Подсчитайте количество заполненных значений в каждом столбце (`".notna().sum()"`).
3. Выведите "DataFrame", оставив только строки, где нет пропущенных значений.

Объясните разницу между `".isna()"` и `".notna()"`.

```
missing_counts = clients_missing_df.isna().sum()
filled_counts = clients_missing_df.notna().sum()
rows_without_missing = clients_missing_df.dropna()

print("Количество NaN в каждом столбце:")
display(missing_counts.to_frame(name="NaN"))

print("Количество заполненных значений в каждом столбце:")
display(filled_counts.to_frame(name="Заполнено"))

print("Строки без пропущенных значений:")
display(rows_without_missing)
```

Рисунок 24. Задание 8

Количество NaN в каждом столбце:

Количество заполненных значений в каждом столбце:

Строки без пропущенных значений:

	NaN
ID	0
Имя	2
Возраст	4
Город	4
Баланс на счете	4
Кредитная история	3

	Заполнено
ID	20
Имя	18
Возраст	16
Город	16
Баланс на счете	16
Кредитная история	17

	ID	Имя	Возраст	Город	Баланс на счете	Кредитная история
0	1	Иван	34.0	Москва	120000.0	Хорошая
7	8	Елена	40.0	Краснодар	175000.0	Хорошая
10	11	Павел	46.0	Омск	250000.0	Хорошая
16	17	Григорий	49.0	Барнаул	275000.0	Отличная
18	19	Степан	30.0	Хабаровск	105000.0	Средняя
19	20	Василиса	35.0	Томск	90000.0	Плохая

Рисунок 24.1 Задание 8

Также было выполнено индивидуальное задание:

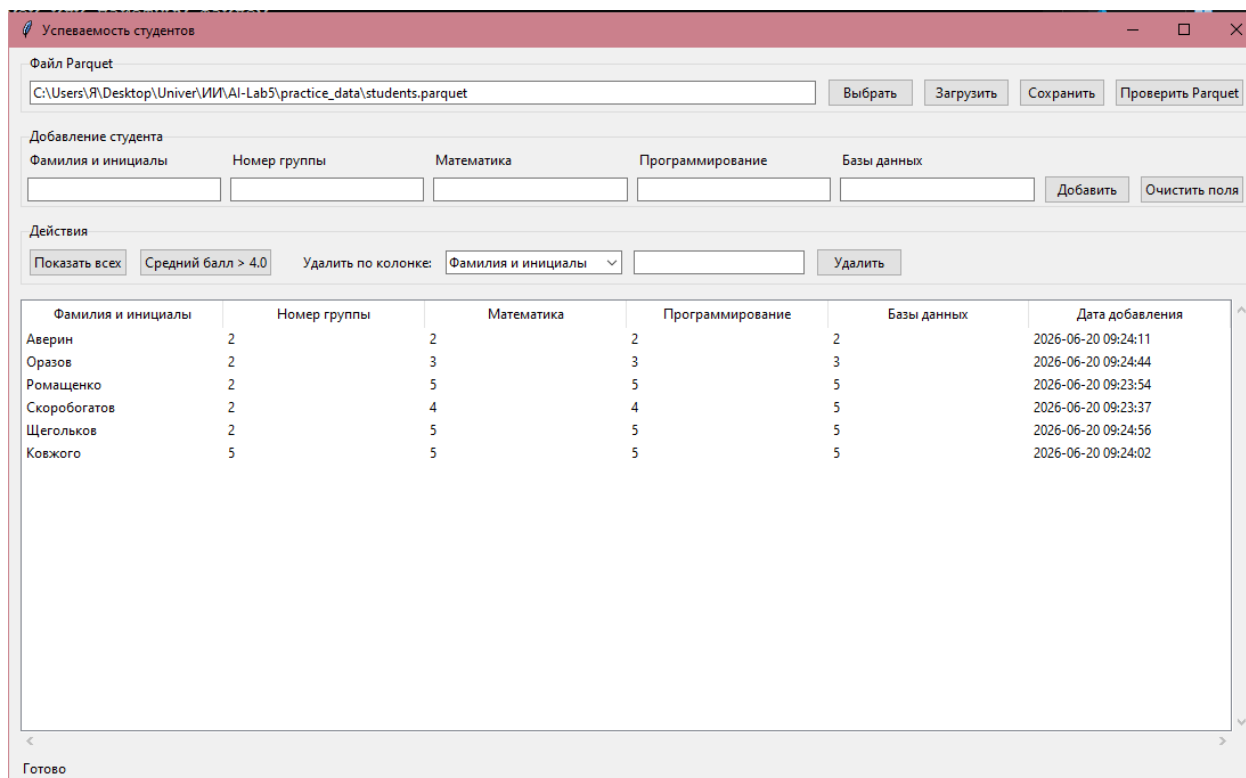


Рисунок 25. Результат выполнения индивидуального задания

Код для программы индивидуального задания приведен в файле Individual_gui.py, в папке practice_data, также там располагаются зависимости, требуемые для её работы.

Ответы на контрольные вопросы:

1. Как создать pandas.DataFrame из словаря списков?

DataFrame из словаря списков создается функцией `pd.DataFrame()`. Ключи словаря становятся названиями столбцов, а списки — значениями этих столбцов. Важно, чтобы списки имели одинаковую длину.

```
import pandas as pd

data = {
    'Имя': ['Иван', 'Ольга', 'Мария'],
    'Возраст': [25, 30, 35],
    'Город': ['Москва', 'СПб', 'Казань']
}

df = pd.DataFrame(data)
```

2. В чем отличие создания DataFrame из списка словарей и словаря списков?

При создании DataFrame из словаря списков каждый ключ соответствует столбцу, а список содержит значения этого столбца. При создании DataFrame из списка словарей каждый словарь обычно описывает одну строку таблицы, а ключи словаря становятся столбцами. Если в отдельных словарях отсутствуют какие-либо ключи, pandas заполнит соответствующие ячейки значением NaN. Словарь списков удобен, когда данные уже организованы по столбцам, а список словарей — когда данные представлены как набор записей.

3. Как создать pandas.DataFrame из массива NumPy?

DataFrame можно создать из массива NumPy, передав массив в pd.DataFrame(). При необходимости через параметры columns и index задаются названия столбцов и индексы строк.

```
import numpy as np
import pandas as pd

array = np.array([[25, 60000], [30, 75000], [35, 80000]])
df = pd.DataFrame(array, columns=['Возраст', 'Зарплата'])
```

4. Как загрузить DataFrame из CSV-файла, указав разделитель ; ?

Для загрузки CSV-файла используется функция pd.read_csv(). Разделитель указывается параметром sep или delimiter. Если в файле данные разделены точкой с запятой, нужно указать sep=';'.

```
df = pd.read_csv('data.csv', sep=';')
```

5. Как загрузить данные из Excel в pandas.DataFrame и выбрать конкретный лист?

Данные из Excel загружаются функцией pd.read_excel(). Конкретный лист выбирается параметром sheet_name. Можно указать название листа или его номер, начиная с нуля.

```
df = pd.read_excel('data.xlsx', sheet_name='Клиенты')

# или по номеру листа
df = pd.read_excel('data.xlsx', sheet_name=0)
```

6. Чем отличается чтение данных из JSON и Parquet в pandas?

JSON — это текстовый формат, удобный для обмена данными и хранения вложенных структур. В pandas он читается функцией `pd.read_json()`, при этом часто нужно учитывать ориентацию данных: `records`, `columns`, `index` и т. д. Parquet — это бинарный столбцовый формат, который лучше подходит для хранения больших таблиц: он быстрее читается и записывается, поддерживает сжатие и хорошо сохраняет типы данных. В pandas Parquet читается функцией `pd.read_parquet()`, но обычно требует установленного движка `pyarrow` или `fastparquet`.

7. Как проверить типы данных в DataFrame после загрузки?

Типы данных в DataFrame можно проверить с помощью атрибута `df.dtypes` или метода `df.info()`. Атрибут `df.dtypes` показывает тип каждого столбца, а `df.info()` дополнительно выводит количество непустых значений и общий объем памяти.

```
print(df.dtypes)
df.info()
```

8. Как определить размер DataFrame (количество строк и столбцов)?

Размер DataFrame определяется атрибутом `df.shape`. Он возвращает кортеж из двух чисел: количество строк и количество столбцов. Количество строк также можно получить через `len(df)`, а количество столбцов — через `len(df.columns)` или `df.shape[1]`.

```
rows, columns = df.shape
print(rows)
print(columns)
```

9. В чем разница между `.loc[]` и `.iloc[]`?

`.loc[]` используется для доступа к данным по меткам индекса и названиям столбцов. `.iloc[]` используется для доступа по целочисленным

позициям строк и столбцов. Например, `df.loc['Мария', 'Возраст']` обращается к строке с индексом 'Мария' и столбцу 'Возраст', а `df.iloc[2, 1]` обращается к элементу в третьей строке и втором столбце. При срезах `.loc[]` обычно включает правую границу, а `.iloc[]` работает как стандартные срезы Python и правую границу не включает.

10. Как получить данные третьей строки и второго столбца с `.iloc[]`?

Так как нумерация позиций в pandas начинается с нуля, третья строка имеет позицию 2, а второй столбец — позицию 1. Для получения одного значения используется `df.iloc[2, 1]`. Если нужно получить результат в виде DataFrame, можно использовать списки индексов: `df.iloc[[2], [1]]`.

```
value = df.iloc[2, 1]
row_as_dataframe = df.iloc[[2], [1]]
```

11. Как получить строку с индексом "Мария" из DataFrame?

Если 'Мария' является индексом DataFrame, строку можно получить через `.loc[]`. Если имя хранится в отдельном столбце, сначала нужно выполнить фильтрацию по этому столбцу.

```
# если 'Мария' — это индекс
row = df.loc['Мария']

# если имя хранится в столбце 'Имя'
row = df[df['Имя'] == 'Мария']
```

12. Чем `.at[]` отличается от `.loc[]`?

`.at[]` предназначен для быстрого доступа к одному конкретному значению по метке строки и названию столбца. `.loc[]` является более универсальным инструментом: он позволяет выбирать строки, столбцы, срезы, списки меток и логические маски. Если нужно получить или изменить одну ячейку, `.at[]` обычно удобнее и быстрее; если нужен диапазон или условная выборка, используют `.loc[]`.

13. В каких случаях `.iat[]` работает быстрее, чем `.iloc[]`?

`.iat[]` работает быстрее, когда нужно получить или изменить одно скалярное значение по числовым позициям строки и столбца. `.iloc[]` поддерживает более сложные операции: срезы, списки позиций, выбор строк и столбцов, поэтому имеет больший служебный overhead. Если в цикле многократно выполняется обращение к отдельным ячейкам по позициям, `.iat[]` может быть эффективнее.

14. Как выбрать все строки, где "Город" равен "Москва" или "СПб", используя `.isin()`?

Для выбора строк по нескольким возможным значениям используется метод `.isin()`. Он возвращает логическую маску, которую затем передают в `DataFrame`.

```
result = df[df['Город'].isin(['Москва', 'СПб'])]
```

15. Как отфильтровать `DataFrame`, оставив только строки, где "Возраст" от 25 до 35 лет, используя `.between()`?

Метод `.between()` проверяет, попадает ли значение в заданный диапазон. По умолчанию границы включаются, то есть будут выбраны значения от 25 до 35 включительно.

```
result = df[df['Возраст'].between(25, 35)]
```

16. В чем разница между `.query()` и `.loc[]` для фильтрации данных?

`.loc[]` использует обычные логические маски `pandas` и является универсальным способом фильтрации. `.query()` позволяет записывать условия в виде строки, что часто делает код короче и ближе к SQL-стилю. Например, `df.query('Возраст > 30')` выглядит компактно. При этом `.loc[]` удобнее для сложных условий, пользовательских функций и динамического построения масок. В `.query()` названия столбцов с пробелами нужно заключать в обратные кавычки.

17. Как использовать переменные Python внутри .query()?

Переменные Python внутри .query() используются с символом @. Это позволяет подставлять значения переменных в строку запроса. Если название столбца содержит пробелы, его нужно заключить в обратные кавычки.

```
city = 'Москва'
min_age = 25

result = df.query('Город == @city and Возраст >= @min_age')
```

18. Как узнать, сколько пропущенных значений в каждом столбце DataFrame?

Количество пропущенных значений в каждом столбце можно определить с помощью цепочки методов df.isna().sum(). Метод .isna() возвращает логическую таблицу, где True означает пропуск, а .sum() подсчитывает количество True по каждому столбцу.

```
missing_count = df.isna().sum()
```

19. В чем разница между .isna() и .notna()?

.isna() возвращает True для пропущенных значений, например NaN, None или NaT. .notna() выполняет обратную проверку и возвращает True для заполненных значений. Эти методы используются для поиска пропусков, фильтрации данных и подсчета заполненных или незаполненных ячеек.

20. Как вывести только строки, где нет пропущенных значений?

Только строки без пропущенных значений можно получить методом df.dropna(). По умолчанию он удаляет все строки, где есть хотя бы один NaN. То же самое можно сделать через логическую маску df.notna().all(axis=1).

```
result = df.dropna()

# альтернативный вариант
result = df[df.notna().all(axis=1)]
```

21. Как добавить новый столбец "Категория" в DataFrame, заполнив его фиксированным значением "Неизвестно"?

Новый столбец добавляется обычным присваиванием по имени столбца. Если присвоить одно фиксированное значение, pandas заполнит им все строки DataFrame.

```
df['Категория'] = 'Неизвестно'
```

22. Как добавить новую строку в DataFrame, используя .loc[]?

Новую строку можно добавить через .loc[], указав новый индекс и значения строки. Значения можно передать списком в порядке столбцов или словарем, где ключи соответствуют названиям столбцов.

```
df.loc[len(df)] = ['Антон', 32, 'Москва']

# вариант со словарем
df.loc[len(df)] = {
    'Имя': 'Антон',
    'Возраст': 32,
    'Город': 'Москва'
}
```

23. Как удалить столбец "Возраст" из DataFrame?

Столбец можно удалить методом .drop(), указав columns. Если нужно сохранить результат, его присваивают обратно переменной. Также можно использовать параметр inplace=True, но чаще рекомендуется явно присваивать результат.

```
df = df.drop(columns=['Возраст'])
```

24. Как удалить все строки, содержащие хотя бы один NaN, из DataFrame?

Все строки, содержащие хотя бы один NaN, удаляются методом dropna() с параметрами axis=0 и how='any'. Эти параметры соответствуют поведению по умолчанию, поэтому достаточно вызвать df.dropna().

```
df_without_nan_rows = df.dropna(axis=0, how='any')
```

25. Как удалить столбцы, содержащие хотя бы один NaN, из DataFrame?

Чтобы удалить столбцы, в которых есть хотя бы один NaN, используется метод `dropna()` с параметром `axis=1`. Параметр `how='any'` означает, что столбец будет удален, если в нем есть хотя бы один пропуск.

```
df_without_nan_columns = df.dropna(axis=1, how='any')
```

26. Как посчитать количество непустых значений в каждом столбце DataFrame?

Количество непустых значений в каждом столбце подсчитывается методом `df.count()`. Он не учитывает NaN-значения. Если нужно посчитать заполненные значения другим способом, можно использовать `df.notna().sum()`.

```
non_empty_count = df.count()

# альтернативный вариант
non_empty_count = df.notna().sum()
```

27. Чем `.value_counts()` отличается от `.nunique()`?

`.value_counts()` показывает, сколько раз каждое уникальное значение встречается в Series или столбце DataFrame. `.nunique()` возвращает только количество уникальных значений, не показывая их частоты. Например, если нужно узнать распределение городов, используют `value_counts()`, а если нужно узнать только число разных городов, используют `nunique()`.

28. Как определить сколько раз встречается каждое значение в столбце "Город"?

Для подсчета частоты каждого значения в столбце используется метод `.value_counts()`. По умолчанию пропущенные значения не учитываются. Если нужно учитывать и NaN, можно указать `dropna=False`.

```
city_counts = df['Город'].value_counts()
```

```
# с учетом пропущенных значений
city_counts = df['Город'].value_counts(dropna=False)
```

29. Почему `display(df)` лучше, чем `print(df)`, в Jupyter Notebook?

В Jupyter Notebook функция `display(df)` выводит DataFrame в виде HTML-таблицы с более удобным форматированием: видны заголовки, индексы, табличная структура и стили отображения pandas. `print(df)` выводит DataFrame как обычный текст, что менее удобно при большом количестве строк и столбцов. Поэтому для анализа данных в ноутбуках чаще используют `display(df)`, а `print(df)` — для простого текстового вывода.

30. Как изменить максимальное количество строк, отображаемых в DataFrame в Jupyter Notebook?

Максимальное количество отображаемых строк задается настройкой pandas `display.max_rows` через функцию `pd.set_option()`. Если указать число, pandas будет показывать не больше заданного количества строк. Если указать `None`, ограничение на количество строк будет снято. Вернуть настройку по умолчанию можно через `pd.reset_option()`.

```
pd.set_option('display.max_rows', 100)

# показать все строки
pd.set_option('display.max_rows', None)

# сбросить настройку
pd.reset_option('display.max_rows')
```

Вывод: в ходе работы были получены знания и навыки работы с основами библиотеки pandas, в частности, со структурой данных DataFrame.